



Unidad 6

Capa de Transporte:
TCP y UDP

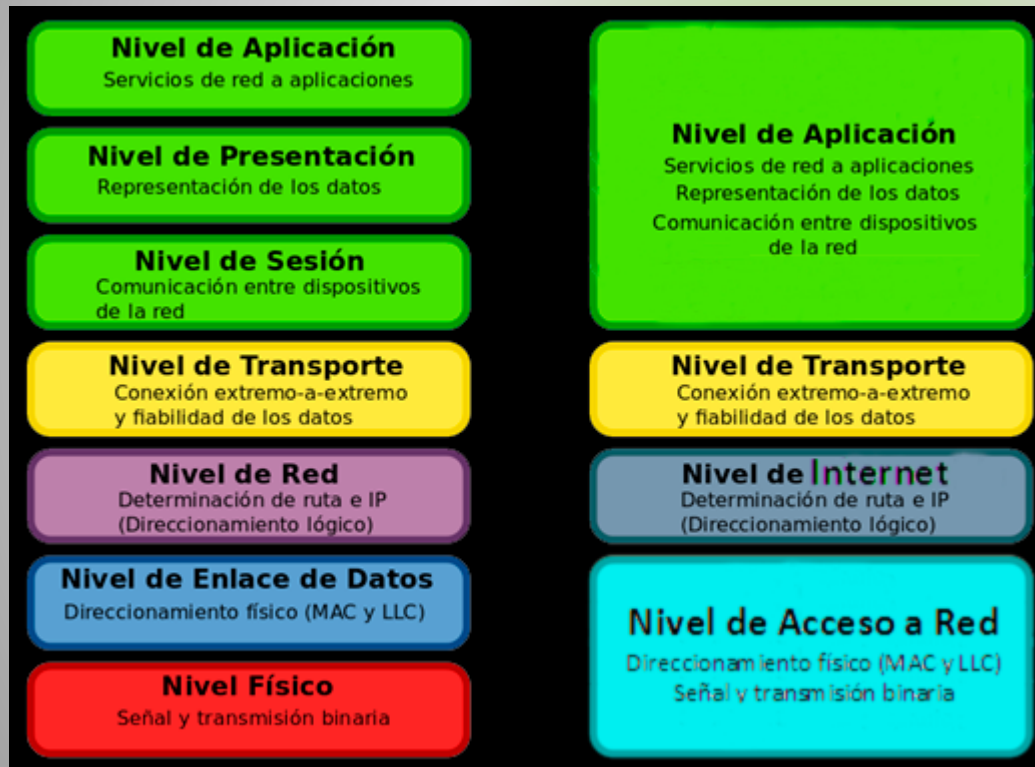
Capa de Transporte

- Introducción
- UDP: entrega sin conexión
- TCP: entrega orientada a la conexión
- Temporización. Ajustes finos
- Extensiones para maximizar performance

Capa de Transporte

- Conectividad EXTREMO-A-EXTREMO entre procesos
- Algunos problemas (y soluciones) ya vistos en enlace: control de flujo, control de errores,...
- Multiplexación de aplicaciones sobre uno/varios enlaces
- Provee conectividad y sincronización entre aplicaciones finales de datos
- Generalmente, la última capa “obligatoria”

Capa de Transporte



Software fuera del S.O.

Software dentro del S.O.

Solo direcciones IP

Solo direcciones físicas

Capa de Transporte

- Debe proveer confiabilidad, abstracción y aislación de problemas inherentes a la subred (variabilidad de retardos, errores de enrutamiento, ...)
- Mucha relación con sistemas operativos. Abstracción basada en IPC
- Debe conectar sistemas finales heterogéneos
- Última capa que realiza control de errores

Capa de transporte en TCP/IP

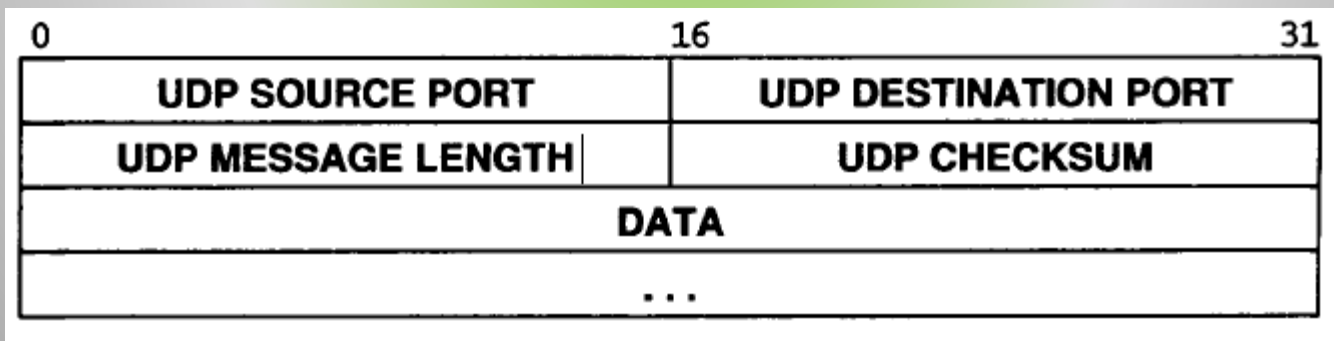
- Problema: ¿cómo encontrar el destino?
- Solución: PUERTOS → abstracción de conexiones lógicas de procesos con la red: SAPs
- Los gestiona el SO. Se dividen en CLIENTES y SERVIDORES. Operación sincrónica c/buffers
- Se asignan mediante la abstracción de SOCKET. Se identifican con la tupla (p, IPo, Po, Ipd, Pd)
- Una vez asignados, identifican unívocamente un proceso (TCP/UDP) en una máquina determinada (IP)

Asignación de puertos

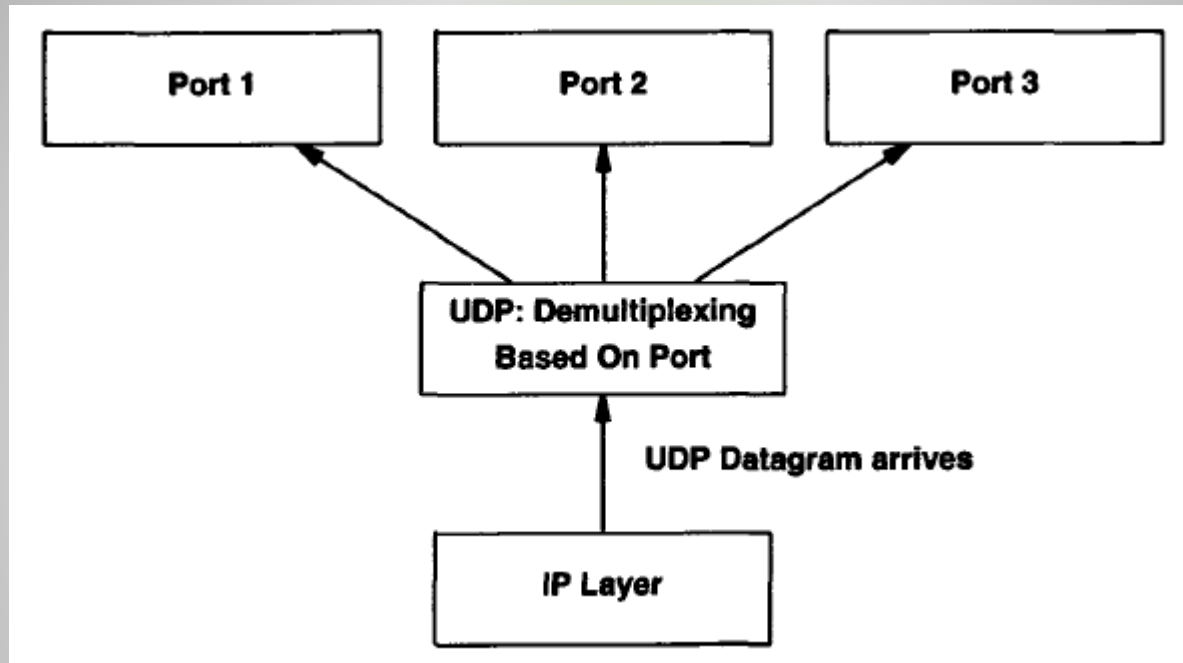
- La IANA define tres rangos de puertos: WKP (0-1023), registrados (1024-49151) y dinámicos o privados (49152-65535)
- El servidor escucha conexiones en los WKP (se puede modificar). PÓR QUÉ?
- Generalmente, los puertos del cliente son asignados por el SO en forma consecutiva a partir del 1024
- Se trata en lo posible de NO REUTILIZAR los puertos del cliente (puertos efímeros)
- Las características de utilización de puertos y persistencia de las conexiones están definidas por las aplicaciones

UDP

- ¿Entrega NO CONFIABLE sobre datagramas NO CONFIABLES ?
- Algunas aplicaciones soportan este modelo de SVC
- Formato del mensaje UDP



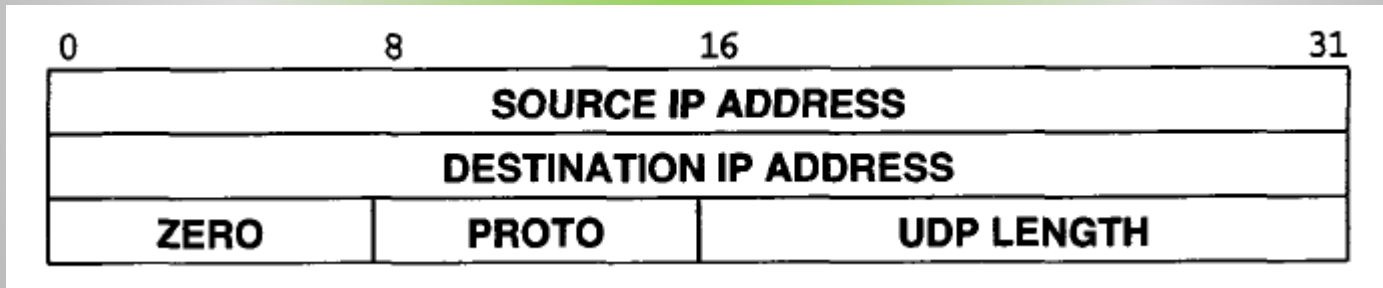
UDP - Multiplexación



Esto permite p.ej. abrir varias conexiones simultáneas con el mismo servidor; usando <> conexiones

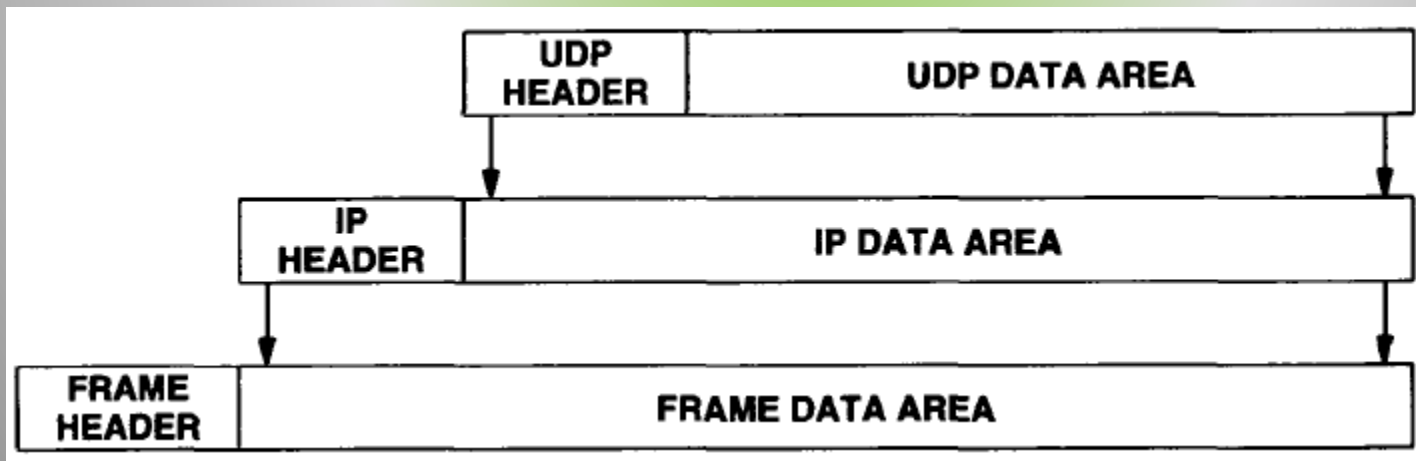
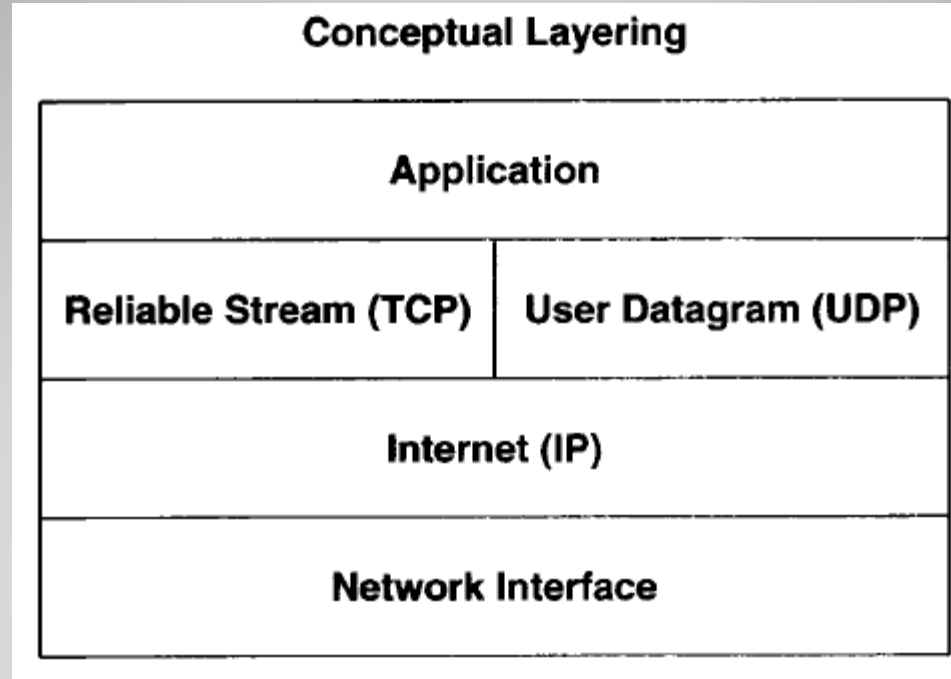
Pseudoencabezado UDP

- PSEUDO ENCABEZADO: se utiliza para incorporar en el cálculo del checksum, 12 octetos DE IP!!!
- ¿PERO..... ESTO **NO VIOLA** LA INDEPENDENCIA DE PROTOCOLOS?
- SI,PERO ESTO ES TCP/IP! (se diseñó así)



- En realidad, se utiliza para ASEGURARSE de que los dg lleguen al destino correcto, ante los errores que se pueden dar en capa 3/4...AUNQUE TCP TAMBIÉN LO USA!!!

UDP – Encapsulamiento. Estratificación



UDP – Puertos bien conocidos (WKP)

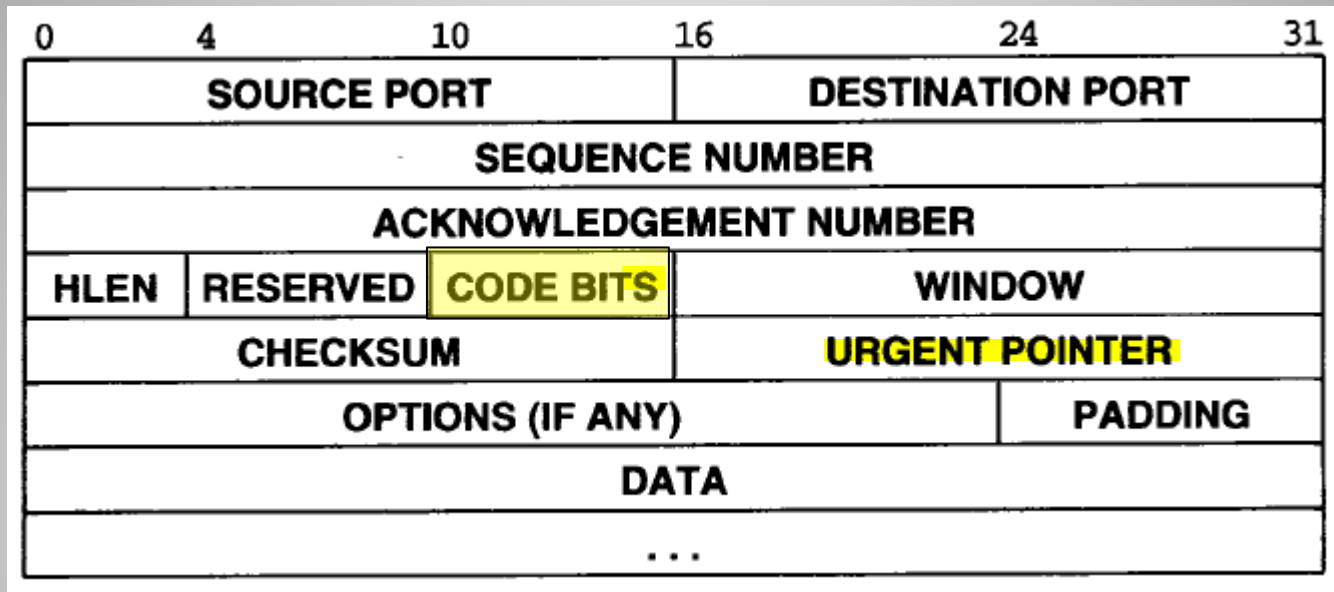
Decimal	Keyword	UNIX Keyword	Description
0	-	-	Reserved
7	ECHO	echo	Echo
9	DISCARD	discard	Discard
11	USERS	systat	Active Users
13	DAYTIME	daytime	Daytime
15	-	netstat	Network status program
17	QUOTE	qotd	Quote of the Day
19	CHARGEN	chargen	Character Generator
37	TIME	time	Time
42	NAMESERVER	name	Host Name Server
43	NICNAME	whois	Who Is
53	DOMAIN	nameserver	Domain Name Server
67	BOOTPS	bootps	BOOTP or DHCP Server
68	BOOTPC	bootpc	BOOTP or DHCP Client
69	TFTP	tftp	Trivial File Transfer
88	KERBEROS	kerberos	Kerberos Security Service
111	SUNRPC	sunrpc	Sun Remote Procedure Call
123	NTP	ntp	Network Time Protocol
161	-	snmp	Simple Network Management Proto
162	-	snmp-trap	SNMP traps
512	-	biff	UNIX comsat
513	-	who	UNIX rwho daemon
514	-	syslog	System log
525	-	timed	Time daemon

TCP

Transmission Control Protocol

- No todas las aplicaciones soportan entrega no confiable (en realidad, muy pocas lo hacen...)
- TCP es MUCHO más complejo que UDP
- Algunas características de funcionamiento de TCP:
 - ◆ Entrega confiable de un flujo no estructurado de octetos. Orientado a la conexión (stream)
 - ◆ Abstracción: VC entre procesos en <> máq
 - ◆ Establecimiento, transmisión, liberación conex.
 - ◆ Full duplex
 - ◆ Utilización de buffers
 - ◆ Números secuencia enumeran cada octeto
 - ◆ Se sigue la tupla (p, IPo, Po, IPd, Pd)
 - ◆ Apertura puertos: Activa → C; Pasiva → S

TCP – Formato encabezado

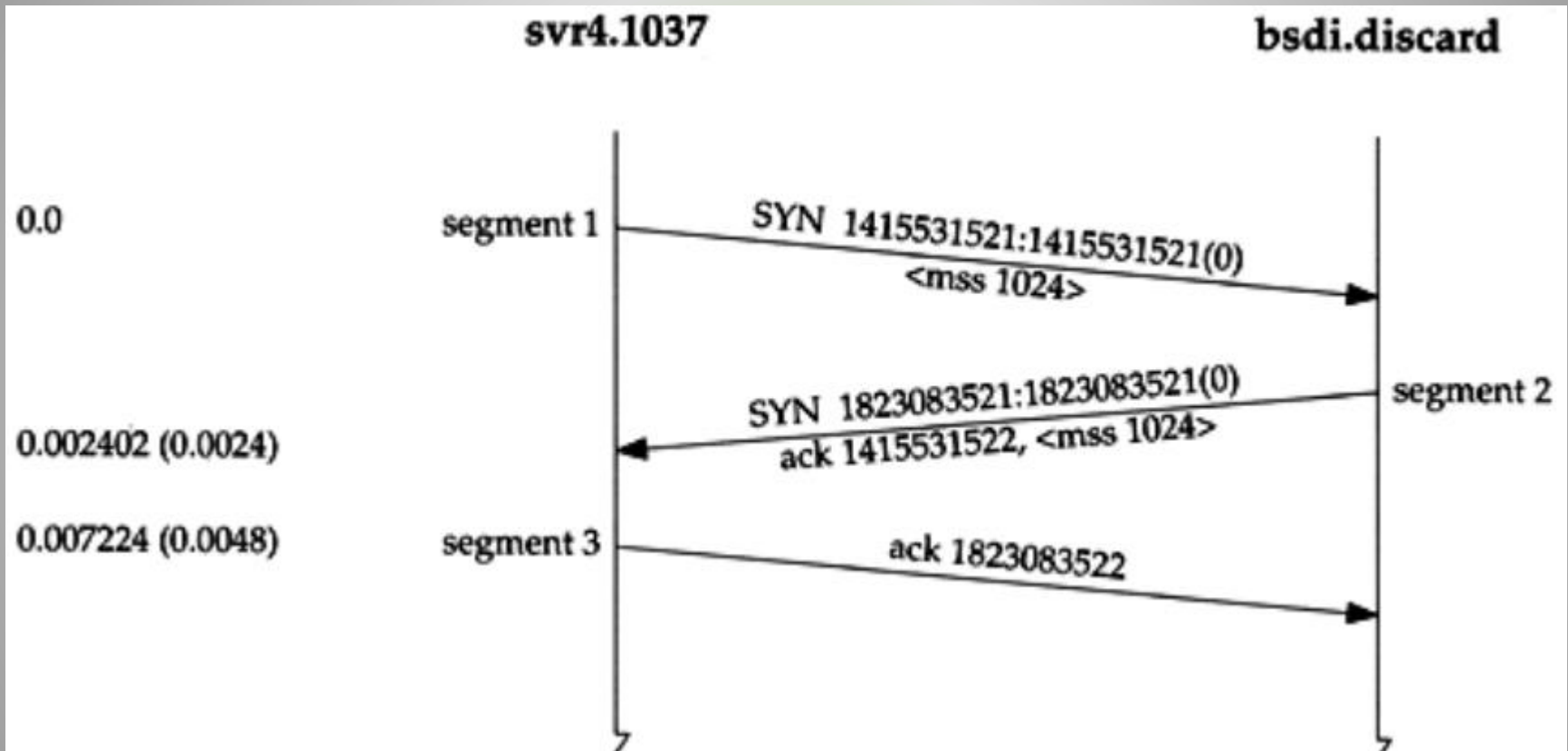


Bit (left to right)	Meaning if bit set to 1
URG	Urgent pointer field is valid
ACK	Acknowledgement field is valid
PSH	This segment requests a push
RST	Reset the connection
SYN	Synchronize sequence numbers
FIN	Sender has reached end of its byte stream

TCP – Números de secuencia

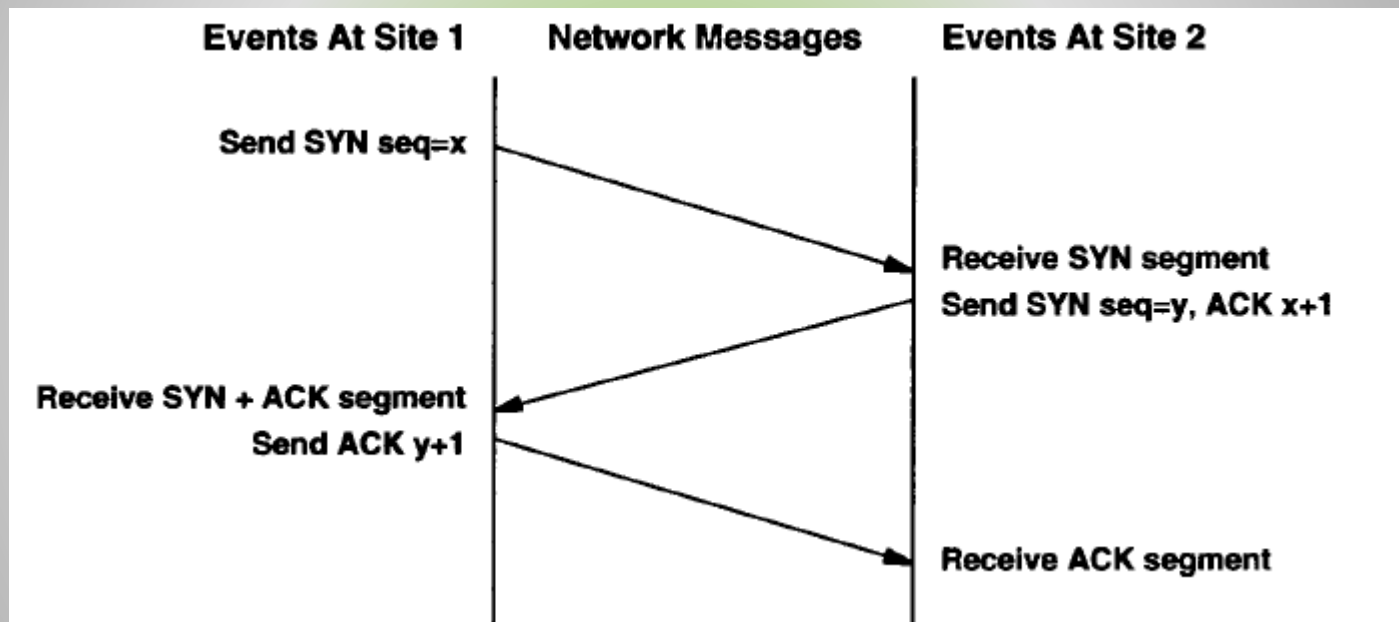
- Son ACUMULATIVOS. Se pueden comparar con un PUNTERO
- Indica la posición del próximo octeto de datos a ser recibido, en el stream de bytes
- Se utiliza uno para cada sentido de la conexión (piggybacking)
- Se asignan inicialmente en forma aleatoria (ISN) Initial Sequence Number
- Ejemplo

TCP – Números de secuencia



TCP – Inicio de la conexión

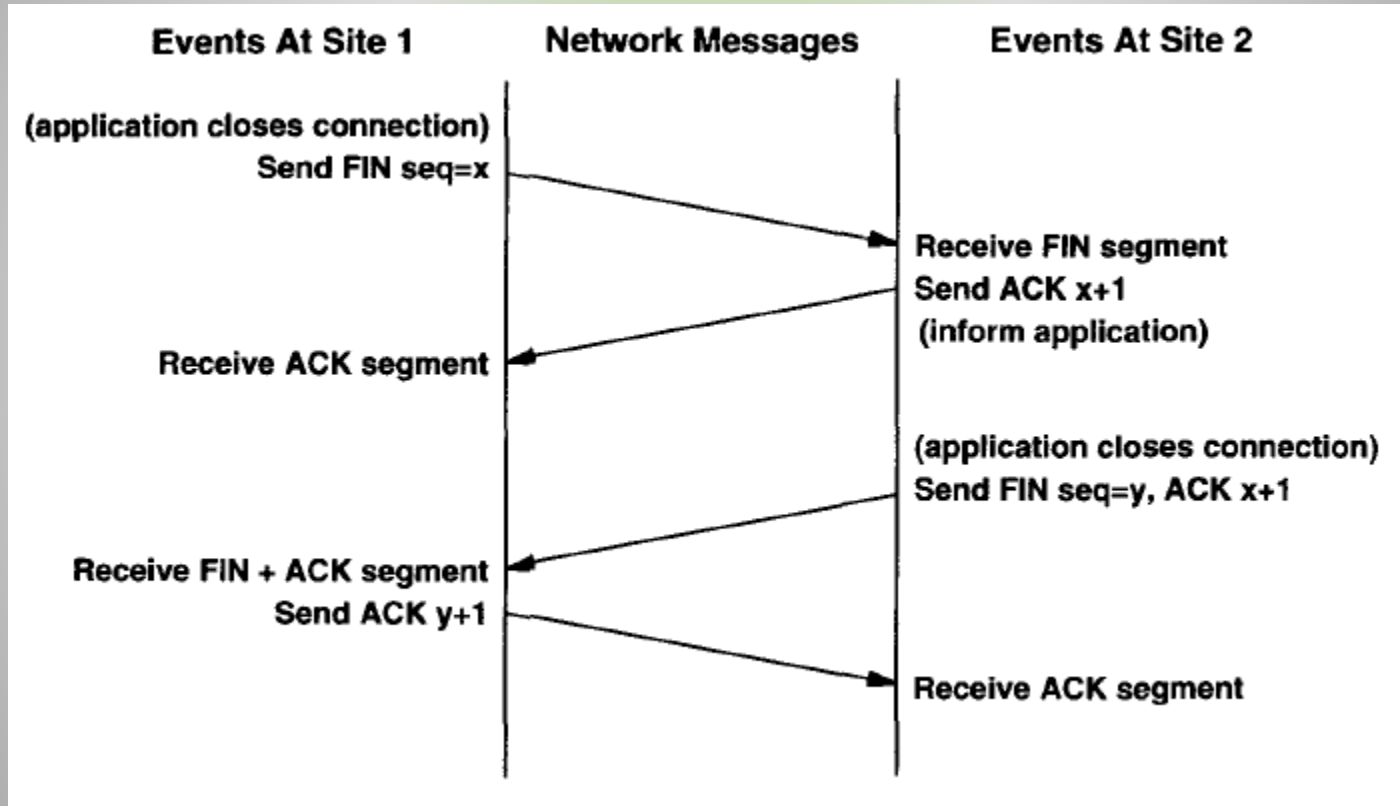
Al establecer la conexión, se emplea el saludo de tres vías:



Se utilizan temporizadores en el caso de no recibir respuesta

TCP – Finalización de la conexión

Al finalizar la conexión, se emplea el saludo de CUATRO vías (para cerrar dos conexiones):



La conexión puede quedar en un estado intermedio (HALF_CLOSE) si Site2 sigue enviando datos.

Frecuentemente se representa mediante una máquina de estados finita donde los nodos representan estados y las transiciones se etiquetan mediante evento/acción.

No siempre que se produzca un evento necesariamente se genera una salida. En este caso sólo cambiamos de estado.

TCP – Puertos bien conocidos (WKP)

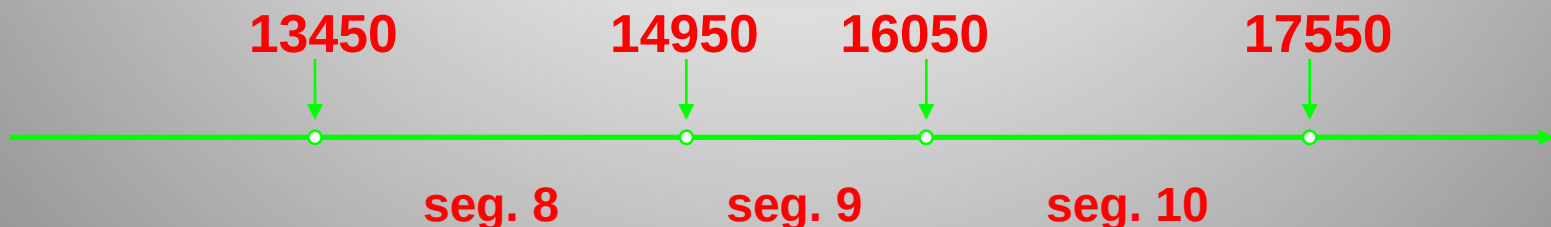
Decimal	Keyword	UNIX Keyword	Description
0			Reserved
1	TCPMUX	-	TCP Multiplexor
7	ECHO	echo	Echo
9	DISCARD	discard	Discard
11	USERS	systat	Active Users
13	DAYTIME	daytime	Daytime
15	-	netstat	Network status program
17	QUOTE	qotd	Quote of the Day
19	CHARGEN	chargen	Character Generator
20	FTP-DATA	ftp-data	File Transfer Protocol (data)
21	FTP	ftp	File Transfer Protocol
22	SSH	ssh	Secure Shell
23	TELNET	telnet	Terminal Connection
25	SMTP	smtp	Simple Mail Transport Protocol
37	TIME	time	Time
43	NICNAME	whois	Who Is
53	DOMAIN	nameserver	Domain Name Server
67	BOOTPS	bootps	BOOTP or DHCP Server
77	-	rje	any private RJE service
79	FINGER	finger	Finger
80	WWW	www	World Wide Web Server
88	KERBEROS	kerberos	Kerberos Security Service
95	SUPDUP	supdup	SUPDUP Protocol
101	HOSTNAME	hostnames	NIC Host Name Server
102	ISO-TSAP	iso-tsap	ISO-TSAP
103	X400	x400	X.400 Mail Service
104	X400-SND	x400-snd	X.400 Mail Sending
110	POP3	pop3	Post Office Protocol Vers. 3
111	SUNRPC	sunrpc	SUN Remote Procedure Call
113	AUTH	auth	Authentication Service
117	UUCP-PATH	uucp-path	UUCP Path Service
119	NNTP	nntp	USENET News Transfer Protocol
123	NTP	ntp	Network Time Protocol
139	NETBIOS-SSN	-	NETBIOS Session Service
161	SNMP	snmp	Simple Network Management Protocol

TCP – Control de flujo

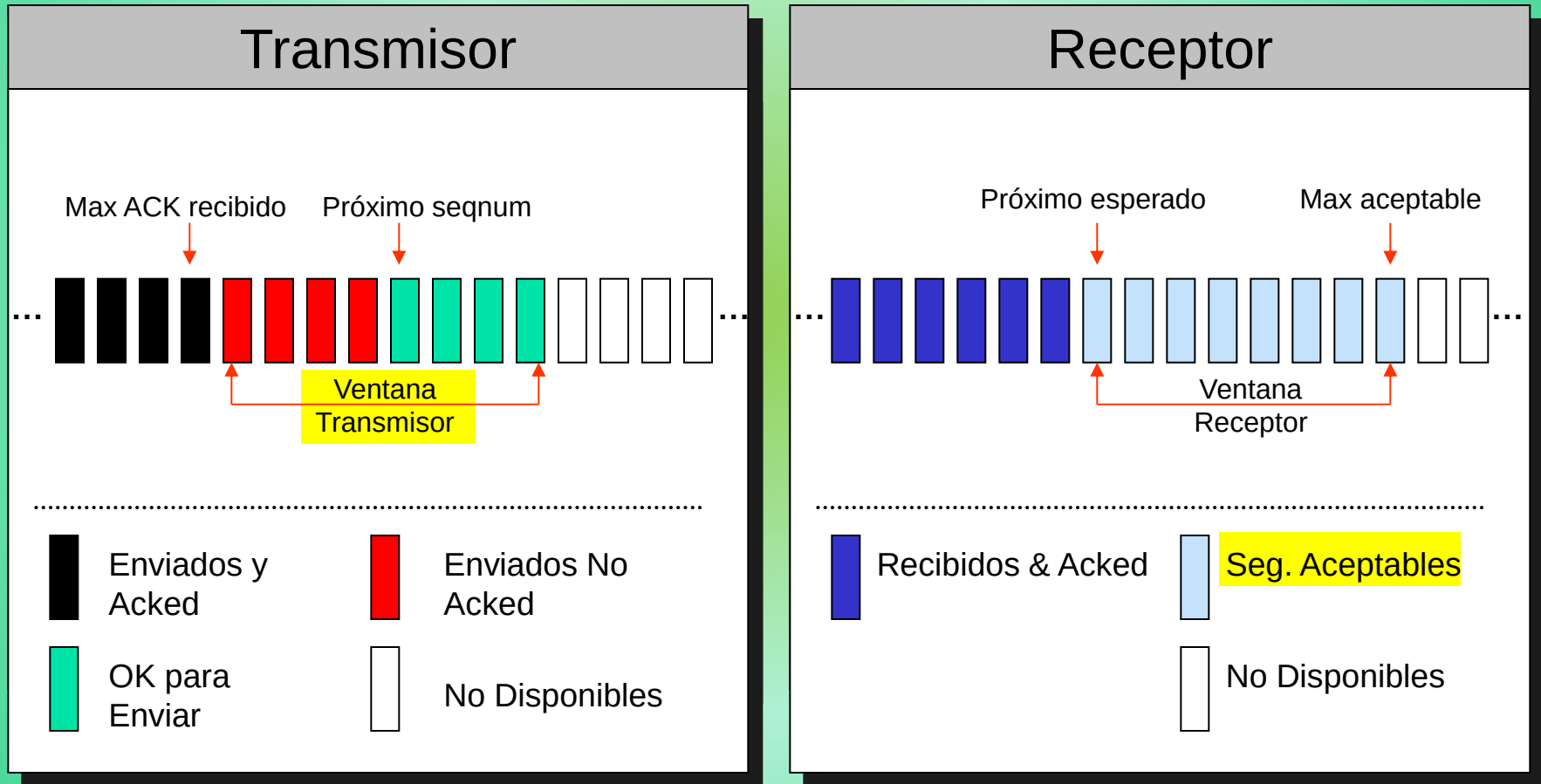
- En cada segmento, el extremo TCP anuncia la cantidad de octetos que pueden quedar pendientes de confirmación
- O sea, la VENTANA del receptor (W)
- Puede ajustarse dinámicamente, si cambian las condiciones en el receptor
- Cuando el receptor no desea recibir más datos, pero mantener abierta la conexión, anuncia una ventana=0

TCP – Número de secuencia

- Número de 32 bits.
- Valor inicial fijado en el establecimiento de la conexión, (ISN).
- TCP desagrega el stream de bytes en segmentos.
 - ◆ Cada segmento tiene asociado el número de secuencia.
 - ◆ Indica su ubicación en el stream de bytes.

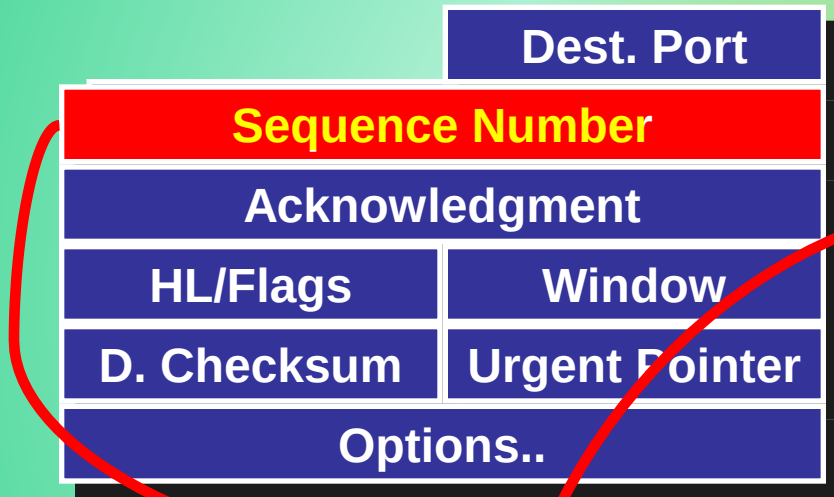


TCP – Ventana deslizando

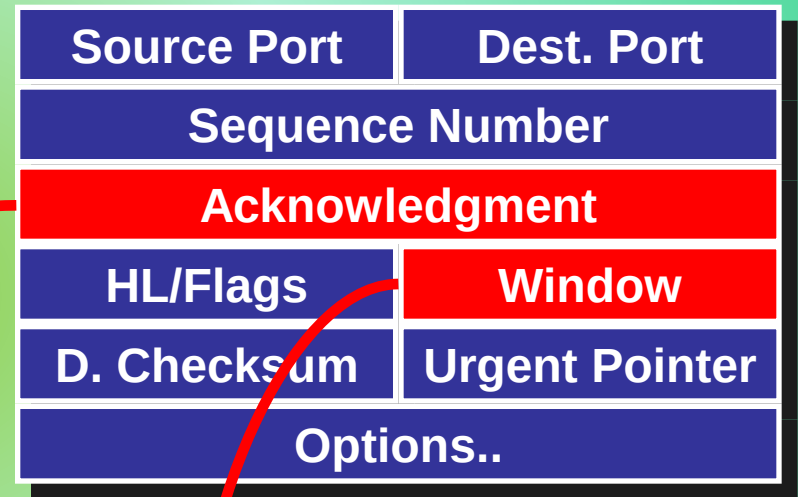


TCP – Ventana deslizable

Segmento enviado



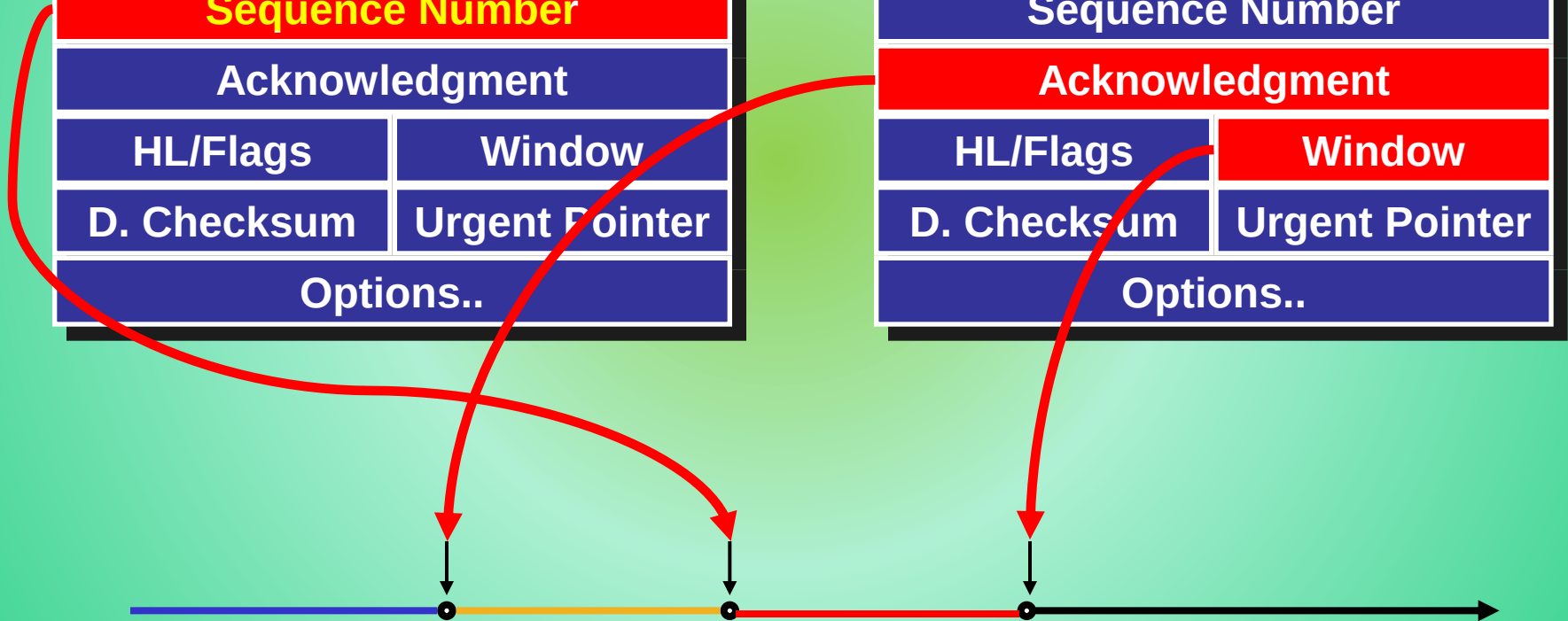
Segmento recibido



acknowledged

enviados

a enviar



TCP – Temporizadores

PARA CADA conexión abierta, CADA EXTREMO TCP mantiene cuatro temporizadores. Ellos son:

- ♦ RTO Timer (1 por cada segmento enviado)
- ♦ 2MSL Timer
- ♦ PERSIST Timer
- ♦ KEEPALIVE Timer

TCP – RTO Timer

- Es el más importante, pues permite adaptarse a las amplias variaciones de retardos en la subred
- Con cada segmento enviado, TCP inicia RTO
- Si llega al límite de tiempo, el segmento se reenvía
- Importante: estimación de RTO
- Múltiplo del RTT;p.ej. $\gamma * RTT$, con $\gamma=2$
- Ahora, ¿cómo estimamos RTT?
- Ejemplo: $NEW_RTT = (\alpha * OLD_RTT) + [(1-\alpha) * NEW_RTT]$
- Algoritmo de Karn y anulación del temporizador
- Especificaciones posteriores desarrollaron métodos estadísticos de estimación de RTO. Ej.

$$DIFF = SAMPLE - Old_RTT$$

$$Smoothed_RTT = Old_RTT + \delta * DIFF$$

$$DEV = Old_DEV + \rho (|DIFF| - Old_DEV)$$

$$Timeout = Smoothed_RTT + \eta * DEV$$

TCP – 2MSL Timer

- Al finalizar una conexión, se exige que la misma pase al estado TIME_WAIT durante un tiempo (2MSL)
- Esto permite que expiren todos los segmentos que pueden haber quedado en la subred - “reencarnación”.
- No se puede reusar durante este tiempo
- Su valor suele definirse en 2 minutos; aunque a veces se implementa con 1 minuto, o 30 segundos.
- RECORDAR!! una conexión TCP está dada por (p, IPo, Po, IPd, Pd)
- Basta con cambiar uno de los parámetros
- Generalmente el puerto del cliente
- Ejemplo:

TCP - 2MSL Timer

C:\WINDOWS\system32\cmd.exe

(C) Copyright 1985-2001 Microsoft Corp.

C:\Documents and Settings\Calcagno>netstat -a -n

Conexiones activas

Proto	Dirección local	Dirección remota	Estado
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING
TCP	10.0.0.104:139	0.0.0.0:0	LISTENING
TCP	10.0.0.104:3234	209.85.195.83:443	ESTABLISHED
TCP	10.0.0.104:3236	209.85.195.83:443	ESTABLISHED
TCP	127.0.0.1:1026	0.0.0.0:0	LISTENING
TCP	127.0.0.1:5152	0.0.0.0:0	LISTENING
UDP	0.0.0.0:445	***	
UDP	0.0.0.0:500	***	
UDP	0.0.0.0:4500	***	
UDP	10.0.0.104:123	***	
UDP	10.0.0.104:137	***	
UDP	10.0.0.104:138	***	
UDP	10.0.0.104:1900	***	
UDP	127.0.0.1:123	***	
UDP	127.0.0.1:1900	***	

C:\Documents and Settings\Calcagno>

TCP – 2MSL Timer

```
C:\WINDOWS\system32\cmd.exe

UDP    127.0.0.1:1900    *:*

C:\Documents and Settings\Calcagno>netstat -a -n

Conexiones activas

Proto  Dirección local      Dirección remota      Estado
TCP    0.0.0.0:135          0.0.0.0:0             LISTENING
TCP    0.0.0.0:445          0.0.0.0:0             LISTENING
TCP    10.0.0.104:139       0.0.0.0:0             LISTENING
TCP    10.0.0.104:3234      209.85.195.83:443     ESTABLISHED
TCP    10.0.0.104:3236      209.85.195.83:443     ESTABLISHED
TCP    10.0.0.104:3245      200.89.148.25:80      TIME_WAIT
TCP    10.0.0.104:3256      190.220.56.34:80      LAST_ACK
TCP    10.0.0.104:3257      190.220.56.34:80      ESTABLISHED
TCP    10.0.0.104:3258      190.220.56.34:80      LAST_ACK
TCP    10.0.0.104:3259      190.220.56.34:80      ESTABLISHED
TCP    10.0.0.104:3260      190.220.56.34:80      ESTABLISHED
TCP    10.0.0.104:3261      190.220.56.34:80      ESTABLISHED
TCP    127.0.0.1:1026       0.0.0.0:0             LISTENING
TCP    127.0.0.1:5152       0.0.0.0:0             LISTENING
UDP    0.0.0.0:445          *:*
UDP    0.0.0.0:500          *:*
UDP    0.0.0.0:4500         *:*
UDP    10.0.0.104:123       *:*
```

TCP – PERSIST Timer

- Como vimos, un extremo TCP puede anunciar una ventana=0 si no desea recibir datos por un tiempo
- ¿Pero cómo puede el otro extremo, saber si aún continúa activo?
- Cuando recibe una ventana=0, inicia un temporizador denominado PERSIST
- Cada vez que expira, se envía una sonda(el último acuse) y si se recibe respuesta positiva, actualiza el temporizador y sigue esperando.
- Se inicializa con el valor del RTO, y se lo duplica en cada prueba.

TCP – PERSIST Timer

- Es necesario hacer una suerte de polling.
- Pueden producirse “**deadlocks**”. Ej: Un receptor envía un ACK con ventana 0, esto congela al emisor de enviar más segmentos. Luego el receptor envía el ACK actualizando la ventana, pero el ACK se pierde. Estamos en presencia de un “deadlock”.
- Al recibirse una ventana de 0 se activa el persist timer. Normalmente 5 segundos.
- Cuando expira se envía un segmento de 1 byte para verificar el estado del receptor.
- El receptor le contesta acorde con el estado en que se encuentra.
- Si el receptor vuelve a contestar con ACK , wind=0.
 - ◆ Entonces el **persist timer** hace un back-off binario.(10, 20, 40seg...).
 - ◆ Al contestar el ACK no validando el byte recibido el emisor continúa enviando este byte de prueba.
- La diferencia con el RTO es que en este caso el emisor envía permanentemente esta prueba hasta que la ventana se incremente o se haga un **reset** de la conexión.

TCP – KEEPALIVE Timer

- Si un extremo TCP no tiene nada para enviar, no envía nada!
- Cuando el otro extremo ignora el estado del proceso, puede decidir enviar una sonda
- Si la respuesta es positiva, incrementa el temporizador y sigue esperando
- Si es negativa, puede dar por terminada la conexión.
- Su valor es de 2 horas! (se puede cambiar)
- Si se dejan de transmitir datos y no se envía el FIN la sesión permanece activa.

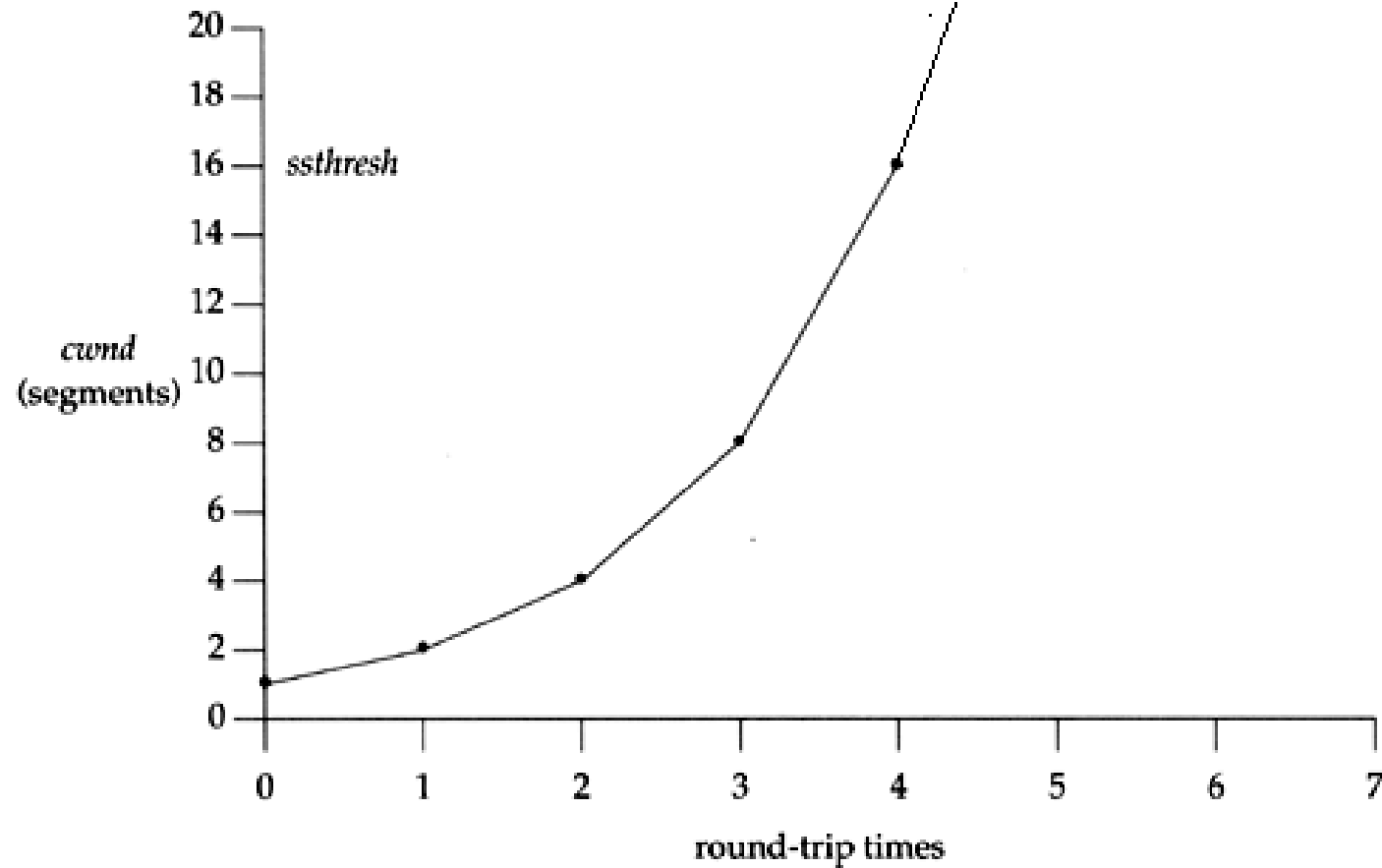
TCP – Refinamientos

- Es necesario utilizar mecanismos para mejorar el rendimiento, cuando la cantidad de información a transmitir es baja (tinygrams). Ejemplo: SWS
- Solución: heurística
- Receptor: acuses de recibo demorados (delayed)
- Emisor: algoritmo de Nagle
- Pueden causar problemas en aplicaciones interactivas.

TCP – Congestión

- Es importante que una nueva conexión TCP inyecte segmentos en forma gradual a la subred.
- De lo contrario, se puede producir congestión
- Para evitarlo, TCP utiliza el mecanismo de arranque lento (Slow Start). Agregamos otro parámetro en cada conexión: la ventana de congestión, o cwnd
- Se inicializa en 1 MSS, y se aumenta en 1 con cada ACK recibido
- Siempre se mide en octetos
- El extremo emisor puede enviar, como máximo, $\min(W, cwnd)$ octetos antes de esperar un acuse
- Ejemplo

TCP – Arranque lento



No es lento, es exponencial!

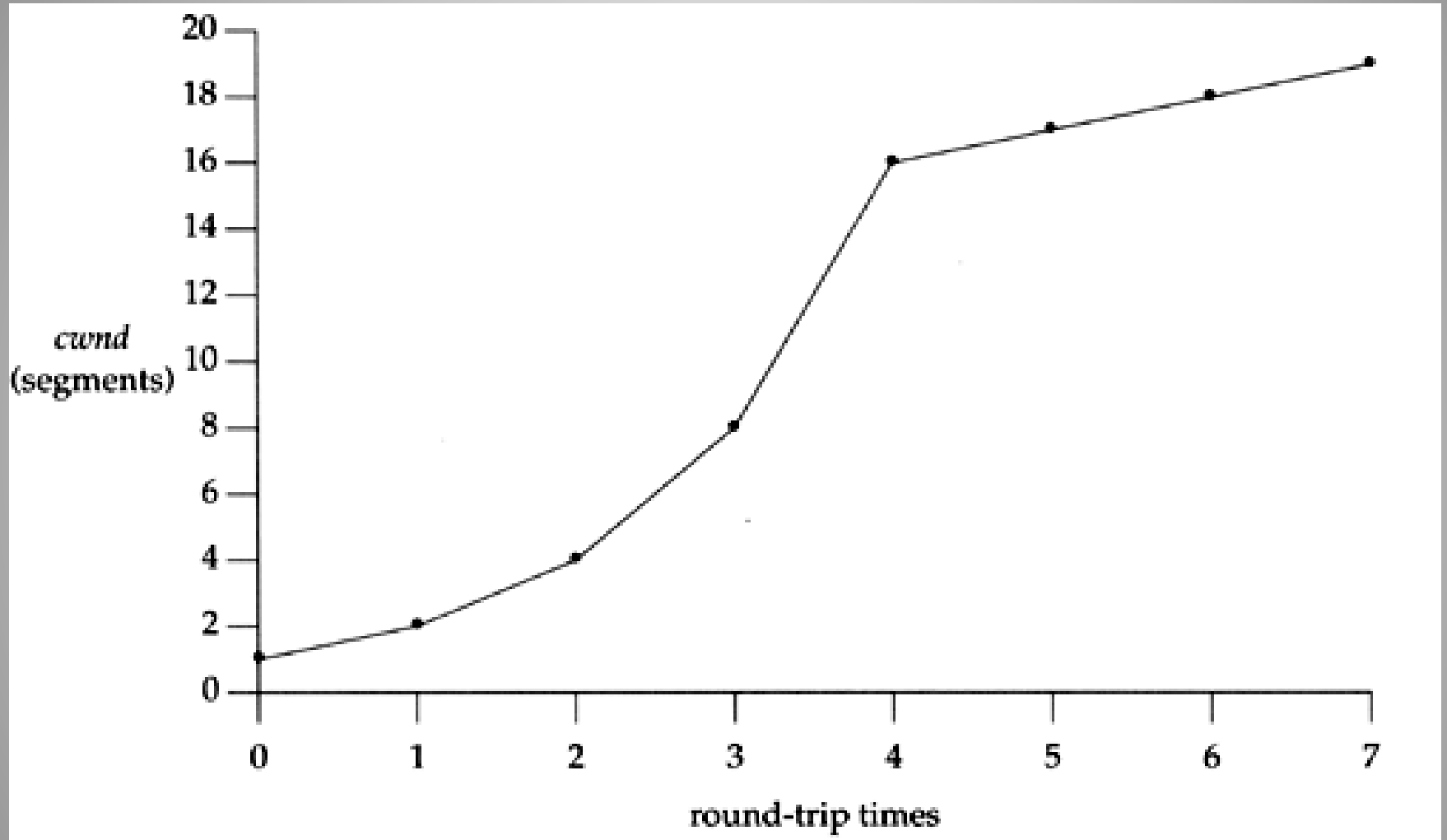
TCP – Congestión

- TCP también interviene activamente en la prevención de la congestión, DURANTE la transmisión de datos (no sólo al comienzo)
- Otra violación de la separación de protocolos???
- Dos eventos indican a TCP que puede existir congestión: RTO (grave) o acuses duplicados (leve)

TCP – Congestión grave (RTO)

- Se utilizan dos fases: Slow Start (ya visto) y Congestion Avoidance
- Se agrega un nuevo parámetro, ssthresh, o umbral de Slow Start. Se inicializa en 65.536
- Este parámetro nos indica hasta cuándo hacemos SS, y cuándo empezamos a hacer CA
- Se aplica SS hasta que ocurra un RTO.
- Entonces, guardamos la mitad de la ventana actual en $ssthresh = \min(W, cwnd)/2$ y $cwnd = 1 \text{ MSS}$
- Iniciamos SS hasta que $cwnd \geq ssthresh$.
- A partir de allí, hacemos $cwnd = cwnd + (MSS * MSS / cwnd)$ con cada ACK recibido correctamente
- Esto incrementa linealmente la ventana, con 1 MSS por cada ronda de ACKs recibidos correctamente
- Ejemplo:

TCP – Ejemplo congestión



Cuánto vale *ssthresh*? Cuánto valía *cwnd*? Qué pasó?

TCP – Congestión leve (TDACK)

- Durante una transmisión, la llegada de varios acuses duplicados puede implicar congestión
- TCP analiza los acuses duplicados, y si encuentra tres seguidos, inmediatamente retransmite el segmento faltante (FAST RETRANSMIT)
- Además invoca el algoritmo de control de congestión, PERO SOLAMENTE la etapa de Congestion Avoidance, o sea la segunda mitad del algoritmo (FAST RECOVERY)
- Ejemplo de acuses duplicados:

slip.1024

vangogh.discard

7.149042

7.419087 (0.0023)

7.420653 (0.0016)

7.688778 (0.0022)

7.778708 (0.0023)

8.226522 (0.4478)

8.228772 (0.0023)

8.496522 (0.2677)

8.498925 (0.0024)

8.500346 (0.0014)

8.766436 (0.2661)

8.768662 (0.0022)

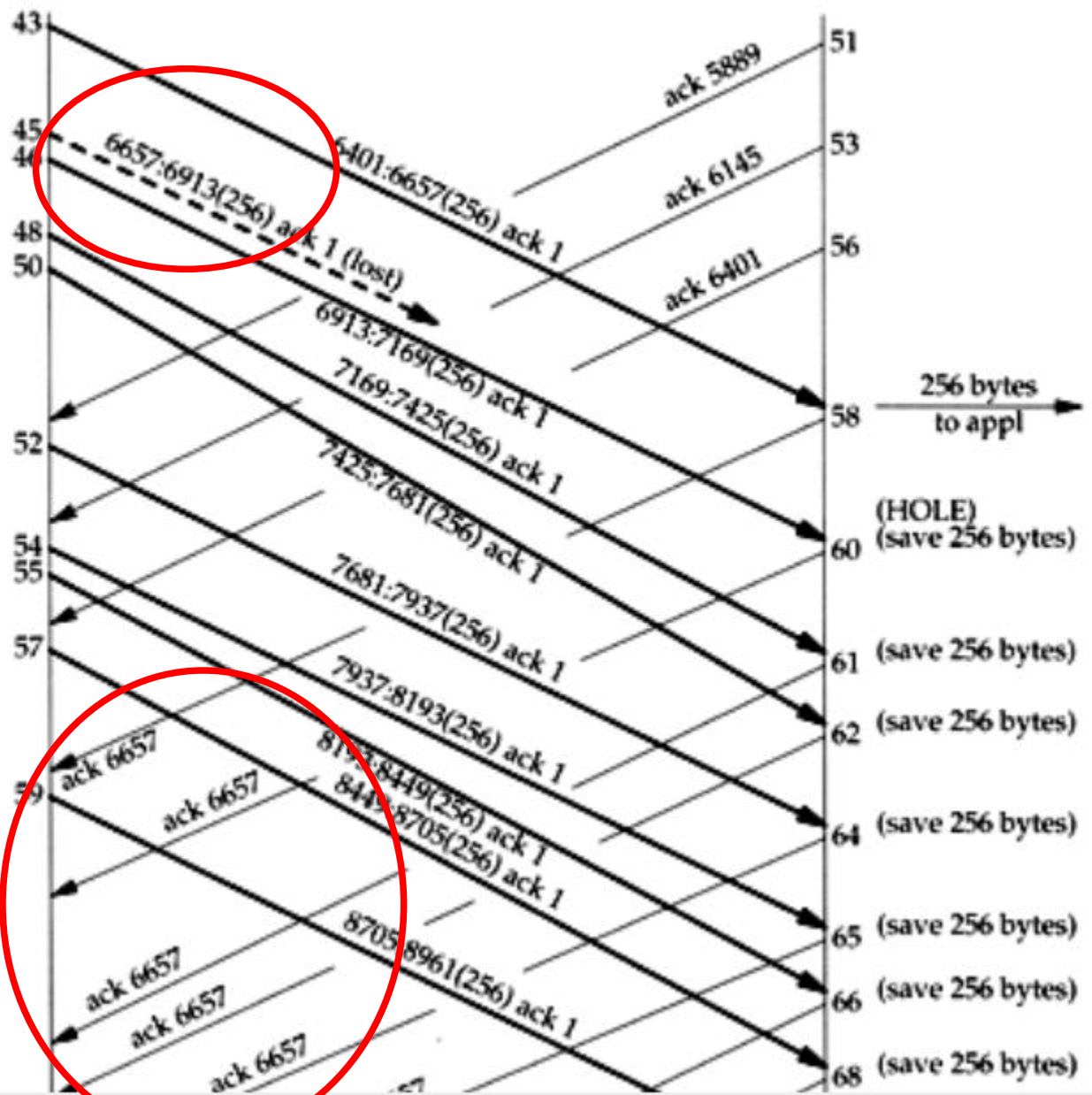
9.156176 (0.3875)

9.158419 (0.0022)

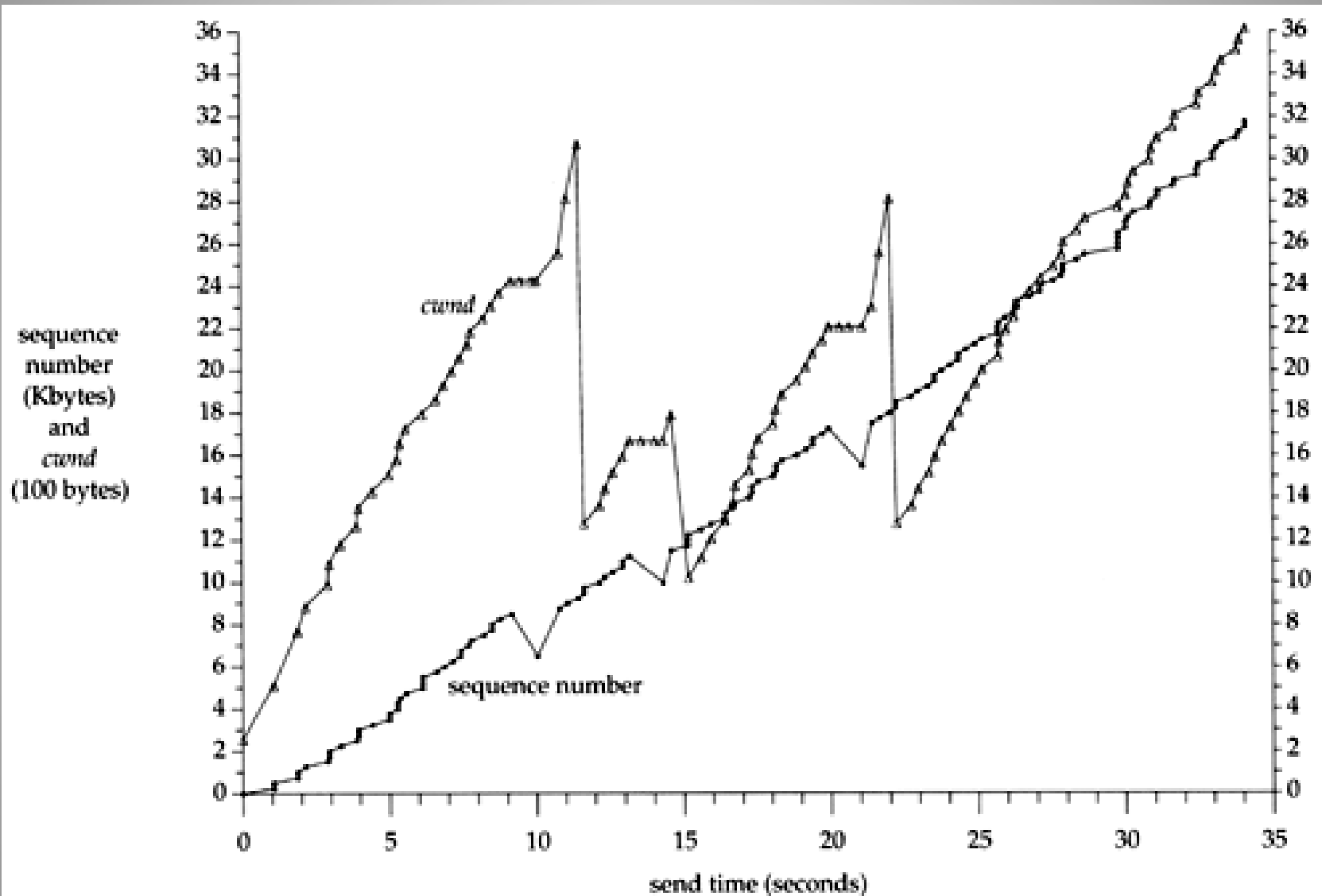
9.489518 (0.3311)

9.879355 (0.3898)

10.000000 (0.1500)



TCP – Ejemplo TDACKs



Congestión en TCP

- Este manejo de la congestión por TCP, impone una visión de “caja negra” de la red.
- Se basa en la expiración de temporizadores.
- Y, más grave, en el descarte de pq/segm
- Esto penaliza muy fuertemente a las aplicaciones sensibles al tiempo (RT)
- Se impone una interacción entre TCP y la subred y también con los mecanismos de QoS de capa 3
 - **Ejemplo: TCP ECN (RFC 3168)**
 - **Más en Unidad 7**
- RECORDAR: Ventana de Congestión (cwnd) es control de flujo impuesto por el emisor; en tanto que Ventana de Transmisión (W) es control de flujo impuesto por el receptor

TCP – Opciones

- TCP soporta un número amplio de opciones
- Aunque muy pocas se utilizan en verdad
- La más utilizada es la de MSS (ya visto)
- Cada extremo utiliza un MSS. No siempre coinciden
- Además, existen opciones para maximizar la performance en redes de gran producto AB-R: las LFN (Long, Fat Networks).ej. Enlaces transcontinentales
- En estas redes, la combinación de grandes anchos de banda y grandes retardos, producen LFP (“caños”) muy grandes, que hay que mantener llenos para optimizar el rendimiento
- ¿Alguna aplicación con estos requerimientos?
- Ejemplos

TCP – Optimizaciones

Network	Bandwidth (bits/sec)	Round-trip time (ms)	Bandwidth-delay product (bytes)
Ethernet LAN	10,000,000	3	3,750
T1 telephone line, transcontinental	1,544,000	60	11,580
T1 telephone line, satellite	1,544,000	500	96,500
T3 telephone line, transcontinental	45,000,000	60	337,500
gigabit, transcontinental	1,000,000,000	60	7,500,000

- W sólo puede contar hasta 64K (opc. Escalamiento)
- Para mejorar la temporización, se puede medir el RTT de cada segmento (opc. Timestamp)
- El número de secuencia puede repetirse en el plazo de una misma conexión (PAWS)

TCP – Escalamiento de ventana

- Para acomodar mayores ventanas, se lleva de 16 a 32 bits el anuncio (W); pero no se cambia el encabezado.
- La opción indica en cuántos bits el emisor ha desplazado el campo W (hasta 14 bits hacia la derecha lo que da una ventana de 1 Gb)
- Para obtener el tamaño real, el receptor vuelve a desplazar en sentido contrario, la misma cantidad de bits
- Se establece en el SYN y no se puede cambiar
- Definido en RFC 1323

TCP – Timestamp

- En TCP estándar, se mide el RTT para un solo segmento de cada W. Si la diferencia entre ambos es muy grande, se introducen fuertes distorsiones.
- El efecto es mayor en las LFNs
- Para evitarlo, se puede medir el RTT para cada segmento
- En esto consiste la opción TIMESTAMP
- También definida en el RFC 1323
- Además es útil para PAWS

TCP – PAWS

- El número de secuencia consiste en un espacio de $2^{32}=4000$ Millones de números.
- Pasado un tiempo, pueden repetirse - o sea “enrollarse” sobre sí mismo (wrap-around) - los números de secuencia.
- Aunque es IMPOSIBLE que se reutilice antes de que expire
- 2MSL. ¿O no?
- 10Mbps → 60'. 45 Mbps (T3) → 12'. 100 Mbps → 5'
- 1Gbps → 34"!! 10 Gbps → 3"!!
- Para detectarlo, usamos TIMESTAMP junto con los nros. de secuencia normales
- DE NUEVO: ¿alguna aplicación ya vista?